CampusSaathi: A Java-Based Web Platform for Academic Assignment Management with Batch-Wise Assignment Access and PDF Question Paper Handling

Manvi Asawa, Mausam, Nikita Rathore, Nikita, Kusum

Department of Artificial Intelligence & Data Science

Arya College of Engineering & I.T., Jaipur, Rajasthan, India

Affiliated to Rajasthan Technical University, Kota

Guide: Er. Ram Babu Buri (Assistant Professor) | Academic Session: 2026–2027 (5th Semester B.Tech in AI & DS)

Abstract*—Assignments and question papers in colleges are often shared through separate channels such as messaging groups, printed sheets and individual file transfers. As a result, students struggle to find the assignment that applies to their batch, and teachers cannot keep assignment details and question papers in one place. This paper presents CampusSaathi, a Java-based web application for academic assignment management built with Java 21, Java Servlets (Jakarta Servlet API), JavaServer Pages (JSP), JDBC and MySQL on Apache Tomcat 11. Teachers create assignments with a name, subject, batch, description and maximum marks, and may optionally attach a PDF question paper. Students log in and see the assignments that belong to their batch, together with status and marks information, and can view or download the attached question paper. The upload feature checks the file size (10 MB limit) and the PDF signature, stores the file under a generated UUID-based name in a server directory, records only that name in the MySQL assignments table, and removes the file if the database insertion fails. A PDF-serving servlet checks the session, the user role and assignment-level permissions before returning a document. A literature survey of 16 IEEE Xplore publications on student planners, student portals, dashboards and usability evaluation reveals six research gaps. We derive a research gap statement, show how the design of CampusSaathi responds to each gap, and define a functional test plan and a System Usability Scale (SUS) evaluation plan. CampusSaathi is implemented as a prototype; formal testing is still to be completed and no measured results are reported. The review is based on abstracts and listing details rather than full texts.

Keywords*—Assignment Management, Student Portal, PDF Question Paper, Batch-Wise Access, Role-Based Access Control, 3-Tier MVC Architecture, Java Servlets, JSP, JDBC, MySQL, Apache Tomcat, Literature Gap Analysis.

I. INTRODUCTION

In a typical college, a teacher announces an assignment in class, shares the question paper through a messaging group or as a printout, and records the marks separately. A student then has to search several chats to find the correct paper, and cannot easily confirm which assignments belong to their own batch or what marks have been recorded. Documents get buried, wrong versions circulate, and there is no single place that connects an assignment to its question paper and its assessment information.

This work presents **CampusSaathi** (tagline: *Your Campus. Connected. Simplified.*), a Java-based academic assignment management system that brings assignment creation, question paper handling, student access and marks-related information into one web application. Before building such a platform it is useful to know what existing student-facing systems already do and what they leave open. We therefore review sixteen IEEE Xplore papers, identify research gaps, and state how CampusSaathi addresses them. The primary contributions are:

- **Centralised Assignment Platform:** A web application on Java Servlets, JSP, JDBC and MySQL where teachers create assignments and students access them from one login-protected portal.
- **Batch-Wise Student Access:** The student page lists the assignments relevant to the logged-in student's batch, with status and marks information.
- **Optional PDF Question Paper Handling:** Multipart upload limited to 10 MB, PDF signature checking, UUID-based filenames, separate file storage, filename saved in the database, and cleanup if the database insertion fails.
- **Role-Aware Document Access:** A PDF-serving servlet that checks the session, the user role and the teacher's or student's right to the specific assignment.
- **Documented Classical Stack:** An implementation on Java 21 and Apache Tomcat 11 using the Jakarta Servlet API that can be reproduced in a college laboratory.
- **Literature Gap Analysis:** A survey of 16 IEEE Xplore papers leading to six research gaps (G1–G6) and a mapped design response.

The rest of the paper is organised as follows. Section II surveys the literature and identifies the gaps, Section III formulates the problem and requirements, Section IV describes the architecture, Section V details the modules, Section VI gives the evaluation plan, Section VII discusses advantages and limitations, Section VIII outlines future scope, and Section IX concludes.

II. LITERATURE SURVEY & RESEARCH GAP ANALYSIS

A. Review Method

Papers were located through web searches aimed at IEEE Xplore, using terms such as *student portal*, *assignment tracking*, *student information system*, *student-facing dashboard* and *system usability scale*. Sixteen papers were selected because each is directly related to at least one aspect of CampusSaathi and was confirmed on its IEEE Xplore listing page. They were grouped as study management [1]–[2], student portals and information systems [3]–[7], dashboards [8]–[13], and usability and technology [14]–[16]. Only abstracts and listing details were read, so a statement that a paper "does not report" a feature means that its abstract does not mention it.

B. Study Management for Teachers and Students

A web application for administration, planning and management of studies, designed from the perspectives of both teacher and student, offers task reminders, a pomodoro timer and a calendar and was refined through a qualitative study with school students [1]. A dashboard design for online learners adds peer progress comparison, deadline reminders and automatically generated to-do lists [2]. Both address planning and deadlines, but their abstracts do not describe teacher-published assignments with attached question papers.

C. Student Portals and Information Systems

A PHP and MySQL student portal provides attendance, assignment tracking, results and quizzes [3]; it is the closest work to CampusSaathi because it includes assignment tracking on a web and database stack, but its abstract does not describe how question documents are stored or protected. Other papers present student information management systems [4], a client/server system with a mobile mini-program [5], and a .NET-based system that classifies student data with a support vector machine and reports an average response time of 0.56 s with no other users online [6]. A student portal combining coding resources with direct event registration is also reported [7]. These systems serve record keeping and administration or bundle assignments with many other functions.

D. Student-Facing Dashboards

A review of student-facing learning analytics dashboards covers 93 articles and calls for studies of design process, needs analysis, usability and measured effects [8]. Dashboards support adviser–student dialogue from grade data [9], real-time feedback in online classes [10], and integration of data from learning management, library and student management systems [11]. Participatory design has yielded design factors for higher-education dashboards [12], and a descriptive dashboard supports inclusive learning environments [13]. All are built on learning analytics or institutional data rather than on simple, batch-targeted assignment access.

E. Usability Evidence and Technology

A usability study of a campus event information system obtained a System Usability Scale score of 70.80, rated only fair, with difficulty navigating the interface [14]. A Java tracker using JDBC and MySQL reports category and date summaries but is a desktop tool [15], and a web-based college event system supports scheduling, organiser access to registration data and notifications [16].

F. Cross-Cutting Findings

The reviewed abstracts rarely cross from general student portals into the specific workflow of a teacher publishing an assignment and a student of the same batch retrieving its question paper. Only [3] explicitly mentions assignment tracking. None of the sixteen abstracts describes file-level authorisation for uploaded question papers. Usability evidence is thin: only [14] reports a standard usability score and [1] a qualitative study, while [8] explicitly asks for stronger usability and needs-analysis studies. The only Java, JDBC and MySQL system found is a desktop application [15].

G. Identification of Critical Research Gaps

- **G1 – Assignment workflow under-described** ([1]–[3]): planners and portals mention tasks or assignment tracking, but not teacher-created assignments that carry subject, batch, description and maximum marks in one record.
- **G2 – Batch-targeted visibility not reported** ([3]–[7]): the abstracts do not describe students seeing only the assignments of their own batch.
- **G3 – Question paper documents and their protection** ([3]–[7]): no abstract describes storing uploaded documents outside the database, validating them, and checking user rights before serving them.
- **G4 – Dashboards are analytics-centric** ([2], [8]–[13]): they present learning data rather than a simple view of assignments, status and marks.
- **G5 – Thin usability evidence** ([1], [8], [14]): standard usability measurement appears only in [14].
- **G6 – Few documented classical web stacks** ([15]): a documented Servlet, JSP, JDBC and MySQL web application with role checks for assignment management is not found, although [15] uses similar technologies in a desktop tool.

H. Proposed Solution Matrix

CampusSaathi responds to each gap through a specific design decision, summarised in Table I.

TABLE I. RESEARCH GAP RESOLUTION MATRIX

Gap	Literature Limitation	CampusSaathi Resolution
G1	Assignment workflow not described in detail.	Assignment record with name, subject, batch, description and maximum marks.
G2	Batch-wise visibility not reported.	Student page filtered by the student's batch.
G3	Document storage and access rules not described.	Optional PDF with validation, UUID filename, separate storage and an access-checking servlet.
G4	Analytics-centric dashboards.	Simple student page with assignment details, status, marks and PDF links.
G5	Thin usability evidence.	Functional test plan and SUS evaluation plan.
G6	No documented classical stack.	Servlets, JSP, JDBC, MySQL on Tomcat 11 with session and role checks.

I. Research Gap Statement

Existing work addresses student planning, student portals, information systems and learning dashboards, but the reviewed abstracts do not describe a lightweight web platform in which teachers publish batch-specific assignments with optional PDF question papers and students retrieve them through a role-checked, session-based portal, with a documented evaluation plan. CampusSaathi is proposed to fill this gap.

III. PROBLEM FORMULATION & SYSTEM REQUIREMENTS

A. Problem Formulation

Let T denote the set of teachers, S the set of students and A the set of assignments. Each assignment a in A is a record (name, subject, batch, description, max marks, creator, optional PDF filename). For a student s in S with batch b(s), the system maintains the following rules:

- **Visibility:** the assignment set shown to s is A_s = { a in A : batch(a) = b(s) }.

- **Upload acceptance:** a PDF p is stored only if size(p) ≤ 10 MB and its content carries the PDF signature; the stored name is a generated UUID-based filename, and only this name is saved in the database.
- **Consistency:** if the database insertion for assignment a fails after the file was written, the stored file is deleted so no orphan file remains.
- **Document access:** a request for the PDF of a is served only if the user is authenticated and either (i) the user is a teacher authorised for a, or (ii) the user is a student with b(s) = batch(a).

The design objective is to give every student one place that shows the assignments, status, marks information and question papers relevant to their batch, and every teacher one place to create assignments and attach papers.

B. Functional Requirements (FR)

- **FR-1:** Teacher and student login with session-based authentication and role checks for protected functionality.
- **FR-2:** Assignment creation by teachers with name, subject, batch, description and maximum marks.
- **FR-3:** Optional PDF question paper upload with a 10 MB limit, PDF signature check, UUID-based filename and storage in a server directory.
- **FR-4:** Student assignment page showing assignments of the student's batch with details, status and marks information.
- **FR-5:** View PDF and Download PDF options when a question paper is attached.
- **FR-6:** Access checks in the PDF-serving servlet for session, role, teacher authorisation and student batch.
- **FR-7:** Existing marks-related functionality on the teacher side, whose information is reflected on the student page.

C. Non-Functional Requirements (NFR)

- **NFR-1 Security:** session checks, role checks and assignment-level authorisation on protected resources; these are to be verified by the test plan, and the prototype is not claimed to be production-grade.
- **NFR-2 Data Integrity:** the database filename and the stored file must stay consistent, including on failure.
- **NFR-3 Usability:** simple navigation for teachers and students; usability to be measured with the System Usability Scale.
- **NFR-4 Maintainability:** controller logic in servlets, presentation in JSP pages, and data in MySQL so that modules can be extended independently.
- **NFR-5 Reproducibility:** a classical stack (Java 21, Tomcat 11, MySQL) with a configurable upload directory.

IV. PROPOSED SYSTEM ARCHITECTURE & DESIGN

A. 3-Tier MVC Architecture

CampusSaathi follows a 3-tier Model-View-Controller organisation (Fig. 1), separating user interface, request handling and data persistence, with uploaded documents kept in a separate server directory.

- **Presentation Tier (View):** JSP pages with HTML and CSS render the login, teacher and student screens.
- **Application Tier (Controller & Logic):** Java Servlets in the package com.campussaathi.controller handle assignment creation, student assignment retrieval and PDF serving (mapped to /assignment-pdf), and apply session and role checks.
- **Data Persistence Tier (Model):** a MySQL database accessed through JDBC stores assignment data, while PDF files are stored separately in a configured upload directory.

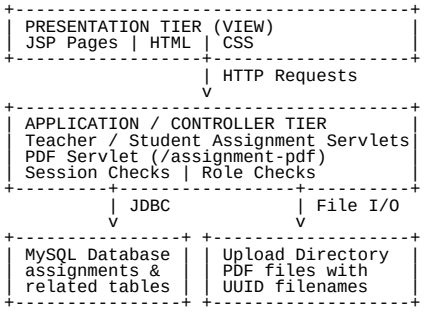


Fig. 1. Three-tier architecture of CampusSaathi.

B. Role-Based Access Matrix

The application distinguishes two roles. Table II shows the intended permission boundaries; each row is to be confirmed by the tests in Section VI before being relied upon.

TABLE II. INTENDED ROLE-BASED ACCESS MATRIX

Capability / Resource	Teacher	Student
Log in to role-specific pages	Yes	Yes
Create assignments	Yes	No
Upload PDF question paper	Yes	No
View assignment list	Authorised ones	Own batch only
View / download PDF	If authorised	If batch matches
Marks-related functions	Manage	View status / marks

C. Relational Database Design

Assignment data is held in the assignments table (Table III). The pdf_filename column was added to support the PDF feature; it stores only the generated filename, while the file itself resides in the upload directory. Authentication and marks-related functionality use further data that is not detailed here.

TABLE III. ASSIGNMENTS TABLE

Column	Type	Purpose
assignment_id	INT, PK, auto-increment	Unique identifier
assignment_name	VARCHAR(100), NOT NULL	Assignment name
subject	VARCHAR(100), NOT NULL	Subject
batch	VARCHAR(50), NOT NULL	Student batch
description	TEXT, nullable	Description
max_marks	DECIMAL(5,2), default 10.00	Maximum marks
created_by	INT, NOT NULL	Teacher identifier
created_at	TIMESTAMP, default now	Creation time
pdf_filename	VARCHAR(255), nullable	Generated PDF filename

V. SYSTEM IMPLEMENTATION & MODULAR BREAKDOWN

CampusSaathi is organised into five connected modules, each presented by one team member, on a shared database. Table IV summarises the technology stack.

TABLE IV. TECHNOLOGY STACK

Tier	Technology	Role
Presentation	JSP, HTML, CSS	Views and screens
Controller	Java 21, Servlets (Jakarta)	Requests, checks, uploads
Data access	JDBC	Database communication
Database	MySQL	Assignment storage
Server	Apache Tomcat 11	Servlet container
Tooling	Eclipse IDE	Development

The application targets Tomcat 11, so the Jakarta namespace (jakarta.servlet.*) is used instead of the older javax.servlet.* namespace.

A. Module 1: Assignment Management (Manvi Asawa)

Teachers create assignments with name, subject, batch, description and maximum marks, and can optionally attach a PDF question paper. Creation and upload are handled by TeacherAssignmentsServlet, which uses multipart form handling.

B. Module 2: Database and Data Management (Mausam)

Assignment information is stored in MySQL and accessed through JDBC. The database stores the generated PDF filename, and the PDF itself is stored in a separate server directory. If the database insertion fails after upload, the file is removed.

C. Module 3: User Interface and Student Portal (Nikita Rathore)

StudentAssignmentsServlet retrieves the assignments of the student's batch with their PDF filenames. The page shows details, status and marks information, and offers View PDF and Download PDF links when a question paper is attached.

D. Module 4: Authentication and Access Control (Nikita)

Teachers and students log in and receive separate experiences based on session and role. AssignmentPdfServlet, mapped to /assignment-pdf, checks the session and role, whether a teacher is authorised for the assignment, and whether a student's batch matches the assignment's batch before serving a file.

E. Module 5: Assessment, Integration and Testing (Kusum)

The existing marks-related functionality is integrated with the student page so that status and marks information appears with each assignment. This module also covers end-to-end workflow checks and the testing described in Section VI.

F. Security and Access Control

Uploads are restricted by size and PDF signature, and stored files receive generated UUID-based names rather than user-supplied names. Protected pages rely on session-based authentication and role checks, and PDFs are served only through the access-checking servlet. These mechanisms are implemented in the prototype, but their effectiveness has not yet been verified by structured tests, and the system is not presented as production-grade secure.

VI. EVALUATION PLAN

The implementation exists as a prototype and the evaluation is in progress; no measured results are reported in this paper. The planned evaluation has two parts, which respond to gap G5.

A. Functional Test Plan

Black-box and boundary tests will be run on each module. Table V lists representative planned cases.

TABLE V. PLANNED BLACK-BOX TEST CASES

ID	Test Scenario	Expected Behaviour
TC-01	Create assignment without PDF	Saved; pdf_filename empty; no PDF links shown
TC-02	Create assignment with valid PDF under 10 MB	File stored with UUID name; name saved in database
TC-03	Upload non-PDF content or file over 10 MB	Upload rejected
TC-04	Database insertion fails after upload	Stored file is removed
TC-05	Same-batch student opens View / Download PDF	PDF is served
TC-06	Different-batch student or unauthorised teacher requests PDF	Access denied
TC-07	Unauthenticated request to /assignment-pdf	Access denied
TC-08	Student attempts a teacher-only action	Action blocked

B. Usability Evaluation

A System Usability Scale (SUS) survey will be administered to student and teacher participants after they complete representative tasks. SUS is chosen because it is the standard instrument reported in [14], allowing comparison with the campus event system whose score was 70.80. Results will be reported once the study is complete.

VII. DISCUSSION, ADVANTAGES & LIMITATIONS

A. Advantages

- **Centralised Access:** assignment details and question papers are available in one portal instead of several channels.
- **Batch Relevance:** students see assignments relevant to their own batch.
- **Separated Storage:** only the filename is kept in the database; the file is stored separately.
- **Controlled Document Access:** PDFs are served through a servlet that checks session, role and assignment permissions.
- **Reproducible Stack:** a classical Servlet, JSP, JDBC and MySQL design suited to DBMS and software engineering coursework.

B. Limitations

- **Review Scope:** the review covers sixteen papers, is not exhaustive and relies on abstracts and listing pages.
- **Evaluation Pending:** functional testing and the SUS study are planned but not yet reported; access restrictions are not yet verified.
- **Scope of Features:** the system does not provide online submission by students, automatic grading, notifications, or attendance and timetable management.

- **Local Storage:** PDFs are stored on the server file system, not in cloud storage.
- **Security Level:** the prototype has not been assessed as production-grade secure, and no automated testing infrastructure is included.

VIII. FUTURE SCOPE & ENHANCEMENTS

- **Student Submissions:** allow students to upload answers and teachers to review them.
- **Reminders:** deadline and new-assignment notifications by email or in-app messages.
- **Cloud Storage:** move uploaded documents to managed cloud storage.
- **Automated Testing:** add unit and integration tests for upload and access rules.
- **Large-Scale Usability Study:** repeat the SUS evaluation across batches and departments.

IX. CONCLUSION

The reviewed literature covers student planners, portals, information systems and dashboards, but the abstracts say little about batch-specific assignment publishing with protected question paper delivery on a classical Java stack. CampusSaathi responds with a Servlet, JSP, JDBC and MySQL application in which teachers create assignments with optional PDF question papers and students access those relevant to their batch through session-based, role-checked pages. The next step is to complete the functional tests and usability evaluation described in Section VI before making stronger claims about reliability and security.

REFERENCES

[1] M. Brožek and M. Dosedla, “Web application for administration, planning and management of studies from the perspectives of the teacher and the student,” in Proc. 22nd Int. Conf. Emerging eLearning Technologies and Applications (ICETA), 2024, doi: 10.1109/ICETA63795.2024.10850733.

[2] “Student-Facing Educational Dashboard Design for Online Learners,” IEEE Xplore, IEEE doc 9251219.

[3] “Development of a Web-Based Student Portal System for University Students,” IEEE Xplore, IEEE doc 10391217.

[4] “Design and Implementation of Student Information Management System,” IEEE Xplore, IEEE doc 5663073.

[5] “Design and Implementation of a Novel Student Information Management System,” IEEE Xplore, IEEE doc 9236801.

[6] “Design and Implementation of Student Management System Based on .NET Architecture,” IEEE Xplore, IEEE doc 10730725.

[7] “Student connect portal using web Application,” IEEE Xplore, IEEE doc 11034801, 2025.

[8] “Review of Research on Student-Facing Learning Analytics Dashboards and Educational Recommender Systems,” IEEE Trans. Learn. Technol., IEEE Xplore, doc 8010828.

[9] “Learning Analytics Dashboards to Support Adviser-Student Dialogue (LISSA),” IEEE Xplore, IEEE doc 7959628.

[10] “Real-Time Learning Analytics Dashboard for Students in Online Classes,” IEEE Xplore, IEEE doc 9368340.

[11] “Capitalizing on Learning Analytics Dashboard for Maximizing Student Outcomes,” IEEE Xplore, IEEE doc 9162357.

[12] “Designing Human-Centered Learning Analytics Dashboard for Higher Education Using a Participatory Design Approach,” IEEE Xplore, IEEE doc 9731917.

[13] “Descriptive Analytics Dashboard for an Inclusive Learning Environment,” IEEE Xplore, IEEE doc 9637388.

[14] “Evaluation of Campus Event Management Information System Using System Usability Scale Method,” IEEE Xplore, IEEE doc 10009978.

[15] “A Study on a Smart Way to track Expenses: from efficiency to effective procurement,” IEEE Xplore, IEEE doc 10074328.

[16] “A Web based-College Event Management System and Notification Sender,” IEEE Xplore, IEEE conf., Coimbatore, Sep. 2022, doc 9985774.